

Data Structure Level 1

Big O Notation

The background of the slide features a complex, abstract network of thin, dark lines crisscrossing a light gray background. In the lower portion of the image, there are dark silhouettes of several people, some standing and some leaning over, suggesting a collaborative work environment or a meeting. A large, dark gray rectangular area is centered on the slide, serving as a backdrop for the text.

Data Structure

Level 1

Big O Notation

Definition: Big O Notation

- ◆ Big O Notation is a mathematical way to describe how the running time or space requirements of an algorithm grow as the input size increases.

 Explanation:

- ◆ It helps measure algorithm efficiency and allows comparison between algorithms.

 Note:

Big O focuses on worst-case growth, not exact execution time.

Explanation: Linear Search and Big O

- ◆ The given program demonstrates Linear Search, which checks elements one by one.

 Result:

Time Complexity = $O(n)$

(Execution time increases linearly with input size)

Definition: Linear Search

◆ A search method that checks each element sequentially until the target value is found or the list ends.

💡 Example:

Searching number 100 inside array:

10 → 20 → 30 → ... → 100

Explanation: Function 1 — FindNumberAlgorithm1

- ◆ Searches entire array even after finding the number.

- Characteristics

- Uses loop through all elements
 - Stores result in variable pos
 - Does not stop after finding number

- Result:

Still $O(n)$ but less efficient practically.

Explanation: Function 2 — FindNumberAlgorithm2

- ◆ Stops searching immediately after finding the number.

- Characteristics

- Returns index directly
 - Avoids unnecessary comparisons
 - Faster in average case

- ➔ Result:

Still $O(n)$ worst case, but faster average performance.



Comparison



FindNumberAlgorithm1

- ✓ Searches entire array
- ✓ Stores result then returns later
- ✓ More unnecessary operations
- ✓ Same Big O but slower practically



FindNumberAlgorithm2

- ✓ Stops when number is found
- ✓ Returns immediately
- ✓ Fewer operations on average
- ✓ More efficient in real execution

Warning

- ◆ In the code, the loop condition is:
`for (int i = 0; i <= n; i++)`

This is dangerous because:

- Array size = 10
- Valid indices = 0 → 9
- Code accesses index 10 →  Out of bounds

 Correct Version Should Be:
`for (int i = 0; i < n; i++)`

Procedures: How Linear Search Works

- 1 Start from first element
- 2 Compare with target number
- 3 If match → return index
- 4 If not → move to next element
- 5 Repeat until end of array

🌟 Summary

- ✓ Big O measures algorithm growth with input size
- ✓ Linear Search has time complexity $O(n)$
- ✓ Early return improves real performance but not Big O
- ✓ Correct loop boundaries are critical in C++
- ✓ Algorithm design + implementation both matter

Thank you 😊